

שאלה 1

בשאלות בהן אתם מבצעים הקצאה דינאמית אין צורך לבדוק שההקצאה הצליחה. בשני הסעיפים בהם אתם צריכים לכתוב קוד אין להשתמש באף פונקצייה מהספרייה הסטנדרטית מלבד הפונקציות להקצאה הדינמית.

סעיף א

כתבו פונקציה בשם `different_numbers` שמקבלת מערך ממויין `arr` של `int` ואת האורך שלו ומחזירה מערך ממויין של המספרים שמופיעים ב-`arr` רק פעם אחת. אורך המערך המוחזר יישמר במקום ה-0, הערכים עצמם יישמרו מהמקום ה-1 והלאה. ניקוד מלא ינתן לפתרון שבו יתבצע מעבר אחד על כל האיברים ב-`arr` ותתבצעה שתי קריאות לכל היותר לאחת מהפונקציות להקצאה הדינמית (`malloc`, `calloc`, `realloc`).

למשל:

- עבור המערך `{1, 4, 4, 6, 7, 10, 10}` הפונקציה תחזיר מערך שיכיל `{4, 1, 6, 7}`
- עבור המערך `{3, 3}` הפונקציה תחזיר מערך שיכיל `{1}`
- עבור המערכים `{1, 1, 3, 3, 4, 5}` הפונקציה תחזיר מערך שיכיל `{3, 4, 5}`.

```
int* different_numbers(int* a, int n) {
    int* new_a = (int*)malloc(sizeof(int)*(n+1));
    bool flag = false;
    int i=1, j=0;

    if(n == 0) {
        new_a[0] = 1;
        return new_a;
    }
    if(n == 1 || a[0] != a[1])
        new_a[i++] = a[0];

    for(int j=1; j<n-1; j++) {
        if(a[j] != a[j-1] && a[j] != a[j+1])
            new_a[i++] = a[j];
    }

    if(n > 1 && a[n-1] != a[n-2])
        new_a[i++] = a[n-1];

    new_a[0] = i;
    new_a = (int*)realloc(new_a, sizeof(int)*new_a[0]);

    return new_a;
}
```

סעיף ב

כתבו פונקציה בשם `common_letters` שמקבלת שתי מילים שמכילות אותיות `uppercase` או `lowercase` בלבד (אין צורך לבדוק) ומחזירה את מספר האותיות המשותפות להן ללא הבדל של `uppercase/lowercase`. ניקוד מלא יינתן לפתרון בו הפונקציה תעבור על כל האותיות בשתי המילים פעם אחת בלבד.
למשל:

- מספר האותיות המשותפות של המילים "Three" ו-"there" הוא 5
- מספר האותיות המשותפות של המילים "Can" ו-"cat" הוא 2
- מספר האותיות המשותפות של המילים "Earth" ו-"Wind" הוא 0

```
int common_letters(char* s1, char* s2) {
    int helper[26] = {};
    unsigned int same = 0;

    for(; *s1; s1++) {
        int index = (*s1>='A' && *s1<='Z')? *s1-'A' : *s1-'a';
        helper[index]++;
    }
    for(; *s2; s2++) {
        char index = (*s2>='A' && *s2<='Z')? *s2-'A' : *s2-'a';
        same += (helper[index]-- > 0);
    }
    return same;
}
```

סעיף ג

נתון הקוד לפונקציה read שקוראת את כל התוכן של הקובץ file_name ומחזירה אותו כמחרוזת. זהו לפחות 5 שגיאות (זמן ריצה, קומפילציה או אלגוריתמיות) בקוד וכתבו אותן. לדוגמא:

- שורה 2: אין בדיקה שפתיחת הקובץ הצליחה, שגיאת זמן ריצה
- שורה 6: אין בדיקה שהקצאת הזכרון הצליחה, שגיאת זמן ריצה
- שורה 10: חסר ; בסוף השורה, שגיאת קומפילציה (כמובן שהשגיאות האלה לא נחשבות כחלק מהחמש שאתם צריכים למצוא...)

```
1. char* read(const char* file_name) {
2.     FILE* fp = fopen(file_name, "r");
3.     char *arr, *p=arr, c;
4.
5.     while(((c = fgetc(fp)) != NULL)) {
6.         arr = (char*)realloc(arr, strlen(arr)+1);
7.         *(p++) = c;
8.     }
9.
10.    return arr
11. }
```

שגיאות:

1. אין סגירה של fp (זמן ריצה)
2. שורה 3: arr לא מאותחל ומכיל כתובת זבל מה שיגרום ל-realloc ליפול (זמן ריצה)
3. שורה 5: fgetc מחזירה EOF בסיום הקריאה ולא NULL. הקוד הזה יגרום ללולאה אינסופית (זמן ריצה)
4. שורה 6: אין הוספה של 0\ בסיום arr מה שאומר היא לא מחרוזת ו-strlen תכשל (זמן ריצה)
5. שורה 7: realloc עשויה לשנות את הכתובת שאליה arr מצביעה כך שצריך לעדכן גם את p. בקוד הקיים זה לא קורה ו-p תצביע לזכרון שנלקח מהתוכנית במקרה כזה (זמן ריצה)

שאלה 2

בשני הסעיפים אין להשתמש באף פונקציה מהספריה הסטנדרטית.

סעיף א

כתבו פונקציה בשם odd_even_bits שמקבלת מספר מטיפוס unsigned int ומדפיסה את המספר שנוצר מהביטים הזוגיים והאי זוגיים שלו בנפרד. למשל:

- עבור המספר 29 שהייצוג הבינארי שלו הוא 11101 הפונקציה תדפיס 2 עבור הביטים האי-זוגיים (10) ו-7 עבור הביטים הזוגיים (111)

- עבור המספר 89 שהייצוג הבינארי שלו הוא 1011001 הפונקציה תדפיס 2 עבור הביטים האי-זוגיים (010) ו-13 עבור הביטים הזוגיים (1101)

```
void odd_even(unsigned int n) {
    unsigned int odd=0, even=0;
    for(int i= sizeof(n)*8-1; i>=0; i--) {
        if(i%2) {
            odd <<= 1;
            odd += ((n & (1 << i)) != 0);
        }
        else {
            even <<= 1;
            even += ((n & (1 << i)) != 0);
        }
    }
    printf("Odd bits number: %u\nEven bits number: %u\n", odd, even);
}
```

סעיף ב

כתבו פונקציה בשם `sort_vals` שמקבלת מערך של מספרים מטיפוס `unsigned int` בטווח של 0 עד 1,600 ללא חזרות ואת אורך המערך `n` (יכול להיות בין 0 ל-1,600) וממיינת אותו. לצורך הפתרון מותר (וגם רצוי) להשתמש במערך של 200 בתים. אין להשתמש במערך ארוך יותר. ניקוד מלא ינתן לפתרון שבו תבצעו מעבר אחד על המערך שהתקבל.

```
void sort_vals(unsigned int* arr, unsigned int n) {
    unsigned char helper[200] = {};
    unsigned int i, j;
    for(i=0; i<n; i++) {
        helper[arr[i]/8] |= (1 << arr[i]%8);
    }
    for(i=0, j=0; i<1600; i++) {
        if(helper[i/8] & (1 << i%8))
            arr[j++] = i;
    }
}
```

שאלה 3

בשאלה זו תממשו מבנה נתונים בשם RoyalSuccession ששומר את סדר הירושה לכתר הבריטי. סדר הירושה מגדיר את רשימת האנשים שנמצאים בתור לכתר הבריטי. מלך אנגליה הנוכחי הוא צ'ארלס השלישי, והראשון בסדר הירושה הוא ויליאם, נסיך וויילס שיהפוך למלך אנגליה לאחר צ'ארלס. הבא בסדר הירושה הוא הנסיך ג'ורג' מוויילס, בנם של ויליאם וקייט מידלטון. שמונת הראשונים בסדר הירושה לכתר הבריטי (כולל המלך הנוכחי) הם

0. המלך צ'ארלס השלישי
1. ויליאם, הנסיך מוויילס (בנם של צ'ארלס ודיאנה)
2. הנסיך ג'ורג' מוויילס (בנם של ויליאם וקייט)
3. הנסיכה שארלוט מוויילס (בתם של ויליאם וקייט)
4. הנסיך לואי מוויילס (בנם של ויליאם וקייט)
5. הנסיך הארי, דוכס סאסקס (בנם של צ'ארלס ודיאנה)
6. הנסיך ארצ'י מסאסקס (בנם של הארי ומייגן מרקל)
7. הנסיכה ליליבאת מסאסקס (בנם של הארי ומייגן מרקל)

נממש את RoyalSuccession על ידי שימוש במודול רשימה מקושרת שהממשק שלו נתון.

```
#ifndef LINKEDLIST_h
#define LINKEDLIST_h

typedef void* Element;
typedef struct LinkedList* LinkedList;

/* Return an empty list. Takes a function that creates a new element (cpy)
   and a function that frees it (fre) */
LinkedList LCreate(Element (*cpy)(Element), void (*fre)(Element));

/* Destroys a list */
void LLDestroy(LinkedList ll);

/* Returns the number of elements in a list */
unsigned int LLSize(LinkedList ll);

/* Adds an element at location index (between 0 to LLSize()). index=0 means
   that the element will be inserted at the beginning of the list, index=1 will
   place the element between the first and second items, and index=LLSize()
   will place the element after the current last element (which will make the
   new element the last one) */
void LLAdd(LinkedList ll, unsigned int index, Element element);

/* Returns the element at index in the list (index between 0 to LLSize-1) */
Element LLGet(LinkedList ll, unsigned int index);

/* Remove and return an element from the list (index between 0 to LLSize-1) */
Element LLRemove(LinkedList ll, unsigned int index);

#endif
```

סעיף א

כתבו ממשק ADT למודול RoyalSuccession והגדירו עבורו את הפעולות הבאות:

- RSCreate: יצירת אובייקט RoyalSuccession
- RSDestroy: הריסת אובייקט RoyalSuccession
- RSGet: החזרת שמו/שמה (כמחרוזת) של מי שנמצא/ת במקום ה-i בסדר הירושה
- RSAdd: הוספת שם חדש (כמחרוזת) במקום ה-i ל-RoyalSuccession. הפונקציה תחזיר true אם ההוספה הצליחה או false אחרת
- RSNumSuccessors: מחזירה את מספר ה(א)נשים בשרשרת הירושה
- RSPrint: הדפסת סדר הירושה באובייקט RoyalSuccession

```
#ifndef ROYALSUCCESSION_H
#define ROYALSUCCESSION_H
#include <stdbool.h>

typedef void* Element;
typedef struct RoyalSuccession* RoyalSuccession;

RoyalSuccession RSCreate();
void RSDestroy(RoyalSuccession);
const char* RSGet(RoyalSuccession, unsigned int);
bool RSAdd(RoyalSuccession, unsigned int, const char*);
unsigned int RSNumSuccessors(RoyalSuccession);
void RSPrint(RoyalSuccession);

#endif
```

סעיף ב

ממשו את ה-struct של RoyalSuccession, את הפונקציה RSCreate וכן כל פונקציה אחרת שנדרשת לצורך יצירת אובייקט RoyalSuccession. יש לשמור את המידע בתוך ה-struct של RoyalSuccession על ידי שימוש ב-LinkedList.

הוסיפו include לכל קובץ h שאתם משתמשים בו.

```
#include "RoyalSuccession.h"
#include "LinkedList.h"
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

struct RoyalSuccession {
    LinkedList royals;
};

Element cpy_str(Element e) {
    return strdup((char*)e);
}

RoyalSuccession RSCreate() {
    RoyalSuccession rs = (RoyalSuccession)malloc(sizeof(struct RoyalSuccession));
    if(!rs) {
        fprintf(stderr, "Error: failed to allocate %u bytes in RSCreate\n",
            sizeof(struct RoyalSuccession));
        exit(1);
    }
    rs->royals = LLCreate(cpy_str, free);
    return rs;
}
```


סעיף ג

ממשו את הפונקציה RSAdd. לפני שמוסיפים שם חדש יש לבדוק האם השם כבר מופיע בסדר הירושה ולהוסיף אותו במקום המבוקש אם לא. אם השם מופיע הפונקציה לא תעשה כלום.

```
bool RSAdd(RoyalSuccession rs, unsigned int index, const char* name) {
    if(index > LLSize(rs->royals))
        return false;

    for(int i=0; i<LLSize(rs->royals); i++) {
        if(!strcmp((const char*)LLGet(rs->royals, i), name))
            return false;
    }
    LLAdd(rs->royals, index, name);
    return true;
}
```